

NiceGUI Testing 模块详解

NiceGUI 的 `testing` 模块是专为自动化测试 NiceGUI 应用设计的核心工具集，旨在模拟用户交互、验证界面状态，让开发者能高效编写单元测试 / 集成测试，确保应用行为符合预期。该模块基于 Playwright 实现底层的浏览器自动化（无需手动配置浏览器），同时封装了贴合 NiceGUI 组件特性的便捷 API，降低测试编写成本。

一、核心定位与依赖

- **核心目标：**模拟用户操作（点击、输入、选择等）、查询组件状态、验证页面渲染结果，覆盖 NiceGUI 应用的前端交互测试场景。
- **底层依赖：**依赖 `playwright` 库实现浏览器自动化，首次使用时会自动下载对应浏览器驱动（Chromium/Firefox/WebKit），无需手动安装。
- **使用前提：**安装 NiceGUI 时需确保包含测试依赖，可通过 `pip install nicegui[testing]` 安装完整依赖。

二、核心 API 与使用流程

1. 基础测试结构

NiceGUI 测试的典型流程为：初始化测试客户端 → 启动应用 → 模拟用户操作 → 断言组件状态 → 清理资源。

```
from nicegui import ui
from nicegui.testing import Screen

def test_basic_button_click():
    # 1. 定义待测试的 NiceGUI 应用逻辑
    click_count = 0
    def increment():
        nonlocal click_count
        click_count += 1

    with ui.column() as main_col:
        ui.button('Click me', on_click=increment, id='test-button')
        ui.label(f'Count: {click_count}', id='count-label')

    # 2. 初始化测试屏幕（模拟浏览器窗口）
    screen = Screen()
    # 3. 启动应用并连接测试屏幕
    screen.start()
    try:
        # 4. 模拟用户操作：点击按钮
        screen.click('#test-button')
        # 5. 断言：标签文本更新
        screen.wait_for('#count-label', text='Count: 1')
        assert screen.find('#count-label').text == 'Count: 1'
    finally:
        # 6. 清理：停止测试屏幕
        screen.stop()
```

2. 核心类：Screen

Screen 是测试模块的核心类，封装了浏览器会话、页面操作和组件查询能力，主要方法如下：

方法 / 属性	功能说明
Screen()	初始化测试屏幕，可指定参数：browser (chromium/firefox/webkit)、viewport (窗口尺寸) 等。
start()	启动测试服务器和浏览器，连接到 NiceGUI 应用。
stop()	关闭浏览器和测试服务器，释放资源。
find(selector)	根据 CSS 选择器 (如 #id、.class) 查找组件，返回 Playwright 元素对象。
wait_for(selector, text=None)	等待组件加载完成 (可选：验证文本匹配)，避免异步渲染导致的断言失败。
click(selector)	模拟点击指定组件 (按钮、链接、复选框等)。
type(selector, text)	模拟在输入框 / 文本域中输入文本。
select(selector, value)	模拟下拉选择框 (ui.select) 选择指定值。
check(selector)	模拟勾选复选框 (ui.checkbox)。
unchecked(selector)	模拟取消勾选复选框。
page	暴露底层 Playwright Page 对象，支持自定义浏览器操作 (如滚动、截图)。
assert_present(selector)	断言组件存在于页面中。
assert_absent(selector)	断言组件不存在于页面中。

3. 关键特性

(1) 异步兼容

NiceGUI 基于异步框架 (FastAPI/Starlette)，testing 模块自动处理异步渲染延迟，wait_for 方法可指定超时时间 (默认 5 秒)，确保组件完全加载后再断言。

(2) 组件标识最佳实践

测试时需为组件指定唯一 id (如 ui.button(..., id='submit-btn'))，避免依赖自动生成的 CSS 类或标签名，提升测试稳定性。

(3) 集成主流测试框架

可与 `pytest` / `unittest` 无缝集成, 示例 (pytest) :

```
import pytest
from nicegui import ui
from nicegui.testing import Screen

@pytest.fixture
def screen():
    screen = Screen()
    screen.start()
    yield screen
    screen.stop()

def test_input_validation(screen):
    # 定义应用
    ui.input(id='email-input', placeholder='Enter email')
    ui.button('Validate', id='validate-btn')
    ui.label(id='error-msg')

    # 测试逻辑
    screen.type('#email-input', 'invalid-email')
    screen.click('#validate-btn')
    screen.wait_for('#error-msg', text='Invalid email format')
    assert screen.find('#error-msg').text == 'Invalid email format'
```

4. 常见测试场景示例

(1) 测试弹窗 (`ui.dialog`)

```
def test_dialog_open_close(screen):
    with ui.dialog(id='test-dialog') as dialog:
        ui.label('Dialog content')
        ui.button('Close', on_click=dialog.close, id='close-btn')
    ui.button('Open Dialog', on_click=dialog.open, id='open-btn')

    screen.click('#open-btn')
    screen.wait_for('#test-dialog', text='Dialog content') # 断言弹窗打开
    screen.click('#close-btn')
    screen.assert_absent('#test-dialog') # 断言弹窗关闭
```

(2) 测试动态组件渲染

```
def test_dynamic_list(screen):
    items = []
    def add_item():
        items.append(f'Item {len(items)+1}')
        list_container.clear()
    with list_container:
        for item in items:
```

```
        ui.label(item, id=f'item-{len(items)}')

list_container = ui.column(id='list-container')
ui.button('Add Item', on_click=add_item, id='add-btn')

screen.click('#add-btn')
screen.wait_for('#item-1', text='Item 1')
screen.click('#add-btn')
assert screen.find('#item-2').text == 'Item 2'
```

三、注意事项

1. **避免 UI 阻塞**：测试中避免使用 `ui.run()`（会阻塞主线程），`Screen.start()` 会自动启动应用的测试服务，无需手动调用 `ui.run()`。
2. **浏览器兼容性**：默认使用 Chromium，如需测试多浏览器，可初始化 `Screen(browser='firefox')` 或 `Screen(browser='webkit')`。
3. **性能优化**：频繁创建 / 销毁 `Screen` 会增加测试耗时，可通过 pytest 夹具（fixture）复用 `Screen` 实例（注意：多用例复用需确保应用状态隔离）。
4. **复杂交互**：对于拖拽、键盘快捷键等操作，可通过 `screen.page` 调用 Playwright 原生 API 实现（如 `screen.page.keyboard.press('Enter')`）。